

Reinforcement Learning



1. Introduction to Reinforcement Learning
2. Markov Decision Processes (MDPs)
3. Policy and Value Functions
4. Dynamic Programming (DP) for RL
5. **Model-Free methods**

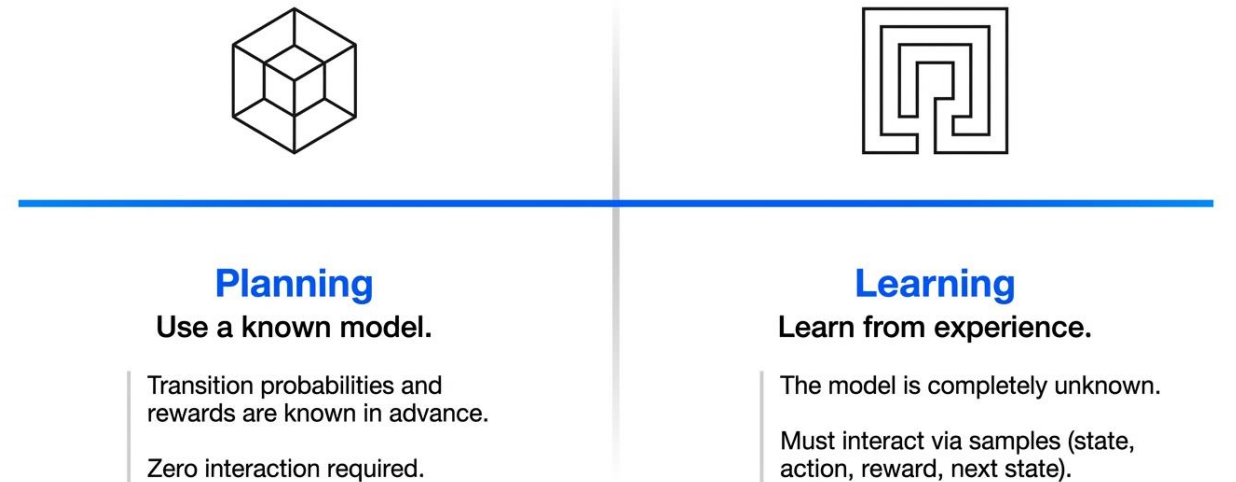
Question 1:

- What is the Goal ?



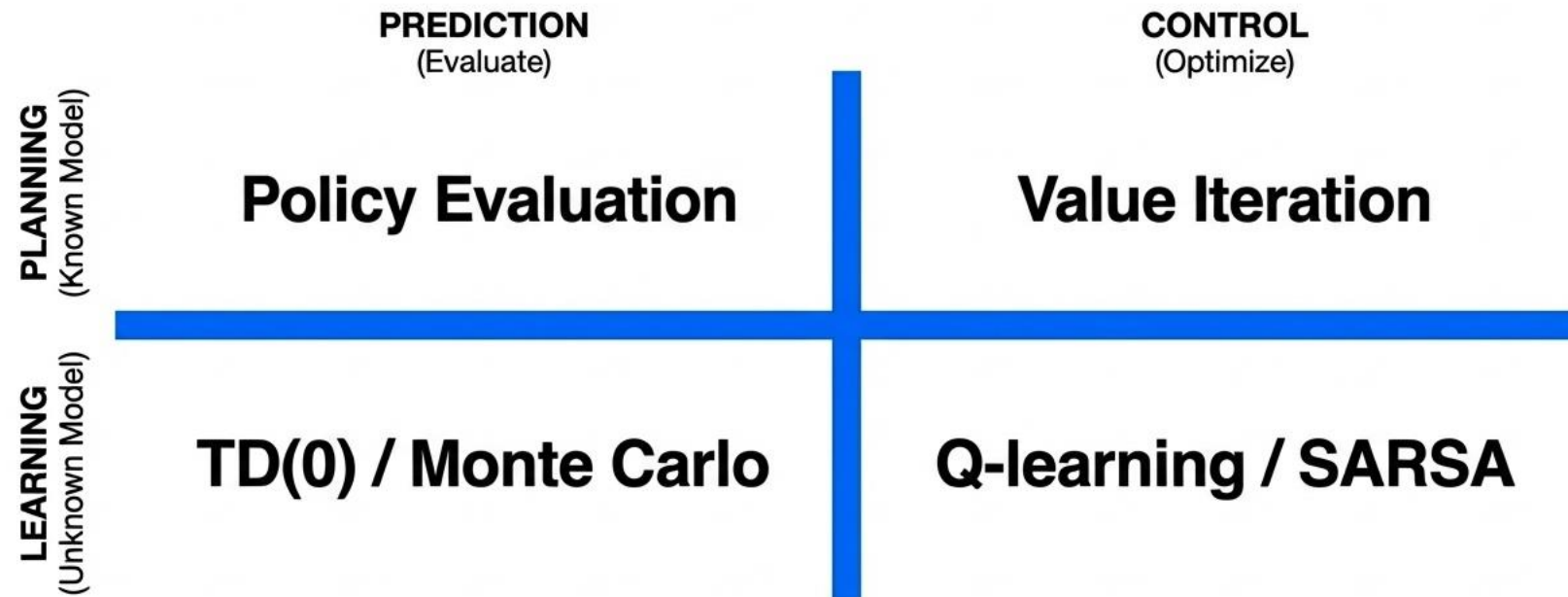
Question 2:

- What information do we have ?



The RL Matrix

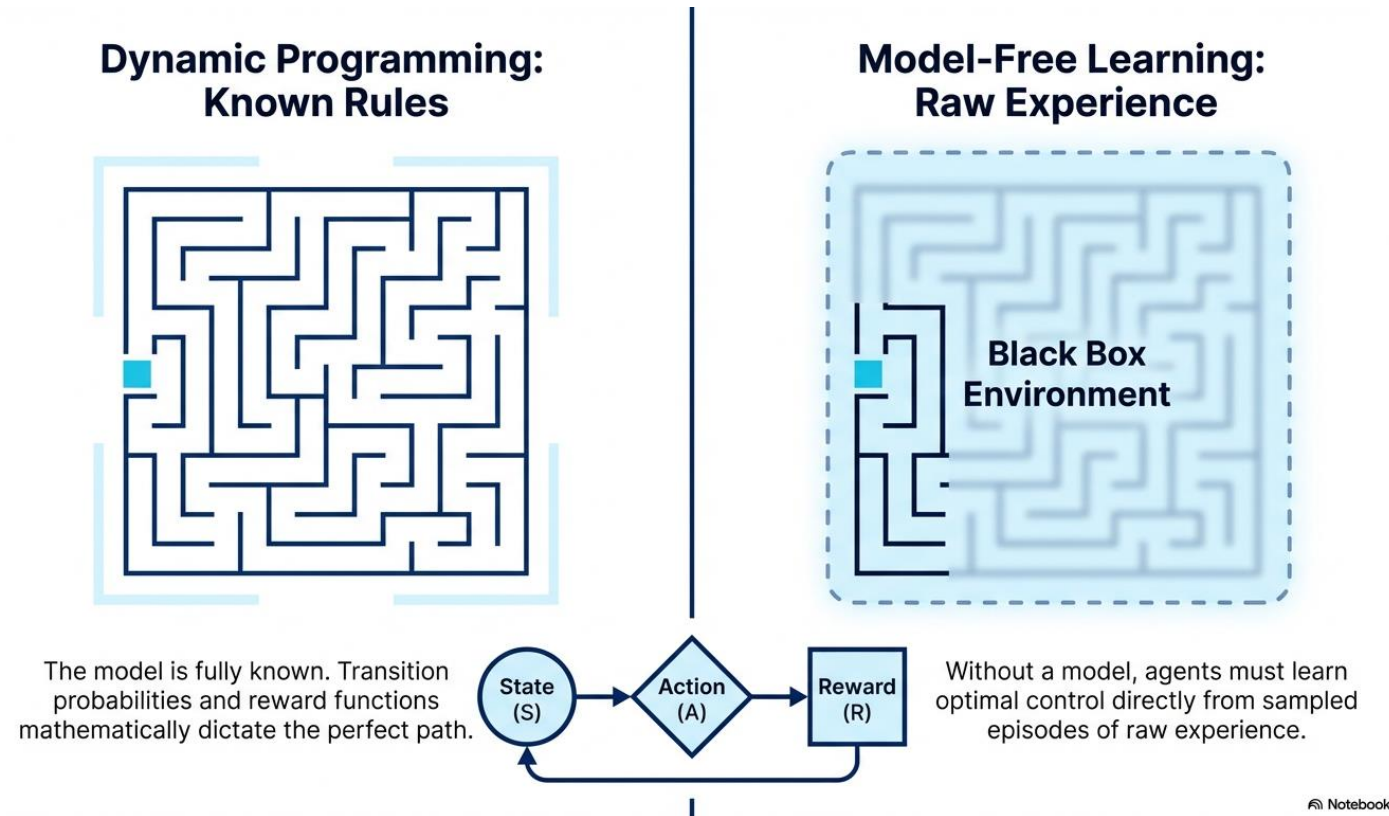
Solving the same problem
under different constraints.



Prediction vs. Control is WHAT we solve. Planning vs. Learning is HOW we solve it.

Bellman Equations: Planning computes expectations exactly; Learning estimates them from samples.

- **What are Model-Free Methods?**
 - Methods that **learn directly from interactions** with the environment.
 - **No explicit model** of the environment (transition probabilities or reward function) is required.
- **Key Characteristics:**
 - **Policy Evaluation:** Learn the value function under a given policy.
 - **Control:** Learn the optimal policy and its corresponding value function.
 - Can be applied to **large or unknown environments** where building a model is infeasible.
- **Applications:**
 - Robotics
 - Game AI
 - Autonomous Systems



1. Monte Carlo (MC) Methods

- Learn from complete episodes of experience.
- Value estimates are based on sample returns.

2. Temporal-Difference (TD) Learning

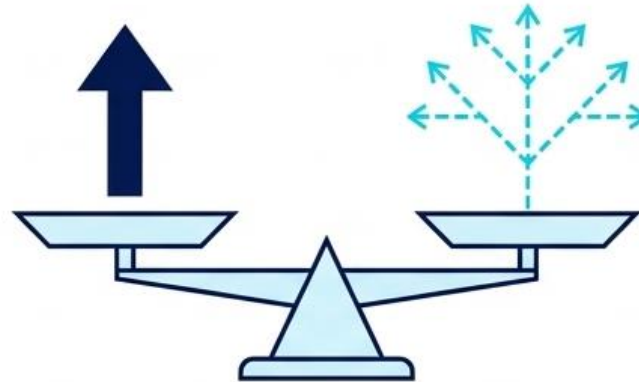
- **Bootstrapping approach:** Updates value estimates using other estimated values.
- **Example algorithms:**
 - TD(0)
 - SARSA
 - Q-Learning

Key Difference:

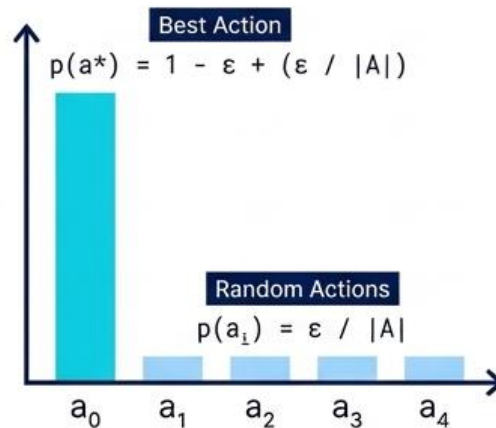
- TD methods can learn online, after every step.
- MC methods require complete episodes for updates.

The Fundamental Dilemma: Exploration vs. Exploitation

Exploitation
Choosing the single action that yields the highest known reward.



Exploration
Trying unknown actions to discover potentially better outcomes.



Probability of best action = $1 - \epsilon + (\epsilon / |A|)$

Probability of other actions = $\epsilon / |A|$

Soft Policies: To ensure continual exploration and mathematical convergence, an agent must use an ϵ -soft policy where every action maintains a non-zero probability of being selected.

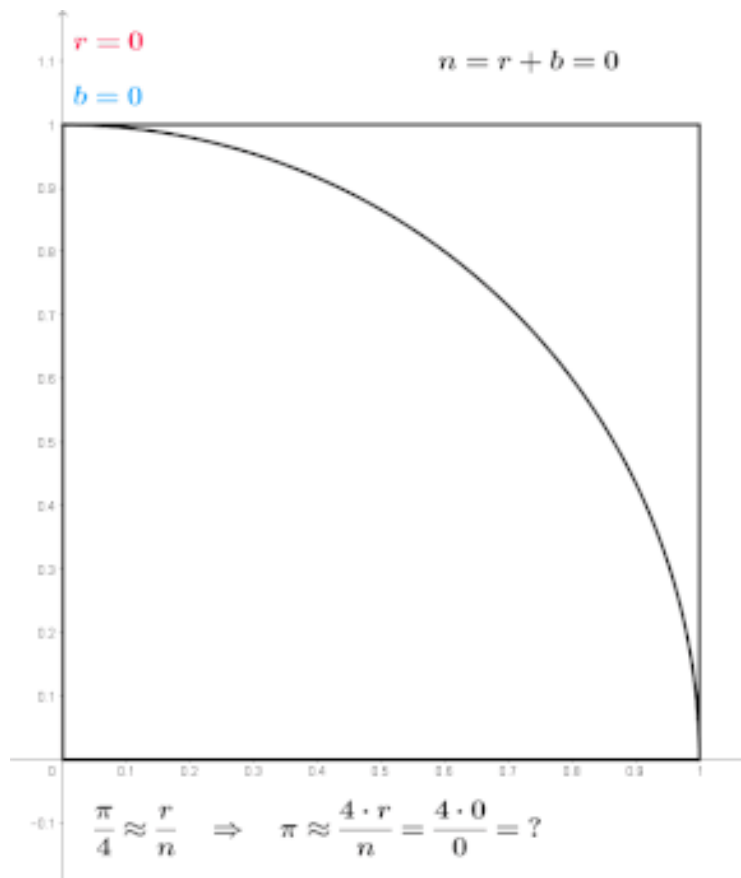
Monte Carlo (MC) Methods

What is Monte Carlo ?

- In general Monte Carlo methods estimate complex quantities by *statistical means*.
- Not specific to RL !

MC in RL :

- A model-free method for estimating value functions
- Relies on sampling from actual experiences (episodes) with the environment
- Used to evaluate and improve policies based on returns



$$\mathbb{E}_{p_{\theta}}[h(X)] = \sum_{i=1}^{\infty} h(x_i) p_{\theta}(x_i)$$

$$\mathbb{E}_{p_{\theta}}[h(X)] = \int_{\mathbb{R}} h(x) p_{\theta}(x) dx$$

$$\frac{1}{N} \sum_{i=1}^N h(x_i) \approx \mathbb{E}_{p_{\theta}}[h(X)]$$

- **Objective:** Estimate $\mathcal{V}_\pi$ based on episodes generated following policy π

$$S_1, A_1, R_2, \dots, S_k \sim \pi$$

- Remember, the return is the cumulative discounted reward:

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$$

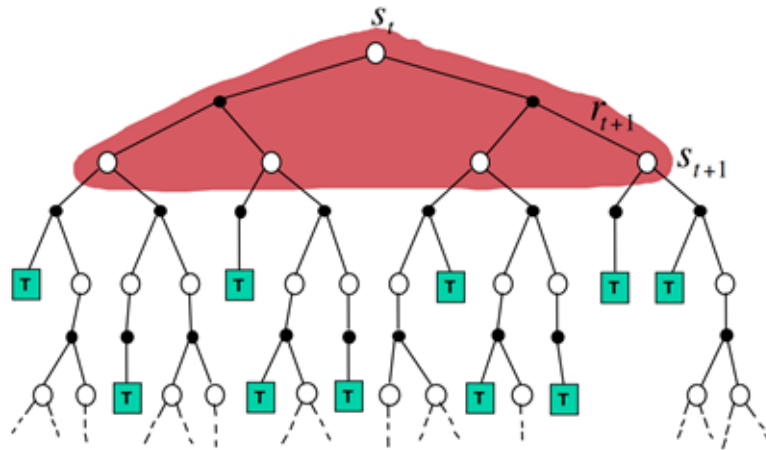
- The value function represents the expected return:

$$v_\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s]$$

- **Monte Carlo policy evaluation** replaces the **expected return** with the **average of observed returns**.

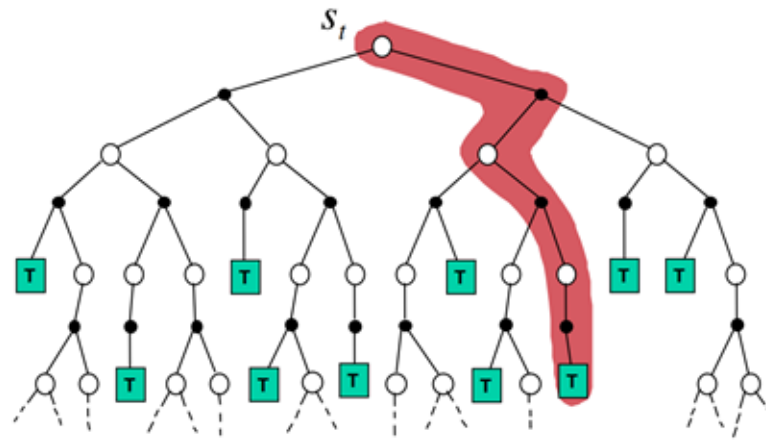
Dynamic Programming

$$V(S_t) \leftarrow \mathbb{E}_{\pi} [R_{t+1} + \gamma V(S_{t+1})]$$



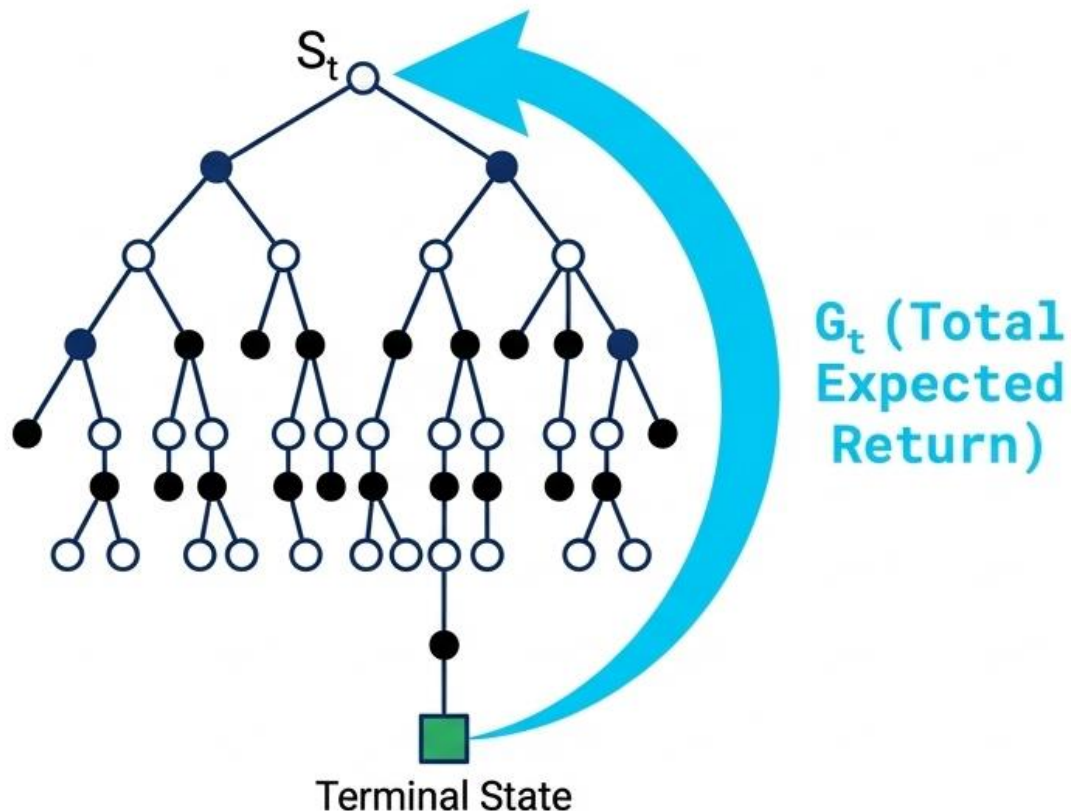
Monte Carlo

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$



Monte Carlo (MC) Prediction

Learning in hindsight at the end of the road.



Wait for the Outcome

MC methods must wait until the episode completely finishes to observe the actual total return.

Average the Returns

The value of a state $V(s)$ is estimated by simply averaging all the sample returns G_t observed following visits to that state.

No Bootstrapping

Estimates are based purely on raw, observed outcomes. We do not update our guesses based on other guesses.

MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

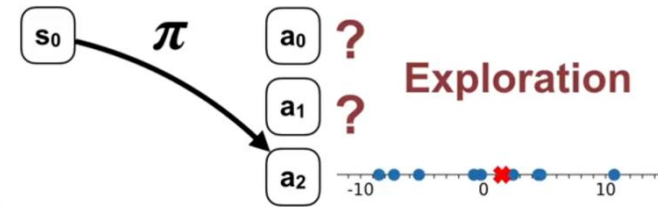
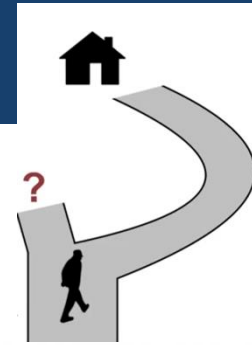
$G \leftarrow \gamma G + R_{t+1}$

Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

Incremental Update

$NewEstimate \leftarrow OldEstimate + StepSize[Target - OldEstimate]$



- If model is known
 - State values can determine a policy
- If model is not known
 - State values only not sufficient to form a policy
 - Estimate of action values needed

- **Problem:** How to estimate $q_{\pi}(s, a)$?
- **Solution:** Use MC to visit state-action pairs instead of states
- **Complication:**
 - Many state-action pair may never be visited (in case of deterministic policy)
- **General problem:** How to maintain exploration?
- **Solution:**
 - **Exploring starts:** the episode start in a random state-action pair (every pair has a nonzero probability)
 - Using stochastic policies (with nonzero probability of selecting all actions in each state)

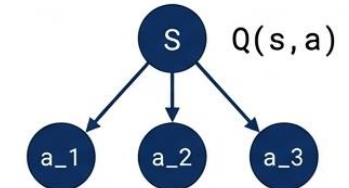
From States to Actions: The Need for Q-Values

State Values



Only useful if you have a known model to predict which action leads to the best next state.

Action Values



Without a model, we must **directly estimate the value** of taking a specific action in a specific state.

The Coverage Problem & Exploring Starts

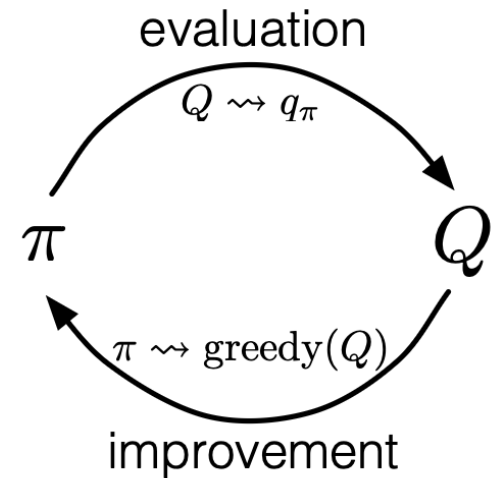
If we follow a deterministic policy, we will only ever evaluate a fraction of possible actions. Many state-action pairs will never be visited.

Assumption: Exploring Starts (ES)

To mathematically guarantee coverage, we theoretically assume every episode begins with a randomly chosen state-action pair with non-zero probability. In reality, we rely on soft policies instead.

- Approximate optimal policies
 - Idea of the GPI
- Policy and Value function creates a moving target for each other
- Together they approach optimality

$$\pi_0 \xrightarrow{\text{E}} q_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} q_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \dots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} q_*,$$



- E: policy evaluation, many episodes experienced (theoretically infinite) with exploring starts
- I: policy improvement, make the policy greedy to the current value function

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:

 $\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$ $Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$ $Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

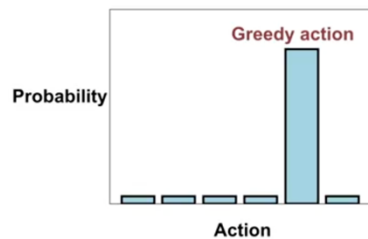
Loop forever (for each episode):

Choose $S_0 \in \mathcal{S}, A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0 Generate an episode from S_0, A_0 , following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$ $G \leftarrow 0$ Loop for each step of episode, $t = T-1, T-2, \dots, 0$: $G \leftarrow \gamma G + R_{t+1}$ Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:Append G to $Returns(S_t, A_t)$ $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$ $\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$

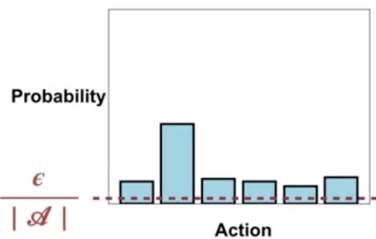
- The overall process converges to the optimal policy and optimal value function
- MC methods can be used to find optimal policies given only sample episodes and no other knowledge of the environment's dynamics
- Unlikely assumptions:
 - Exploring starts (random (s,a) could be dangerous when learning from actual interaction)
 - It can be difficult to randomly sample an initial State action pair.
 - For example, how would you randomly sample the initial State action pair for a self-driving car?
- ϵ -soft policies force the agent to continually explore

ϵ -Greedy Exploration

ϵ -Greedy policies

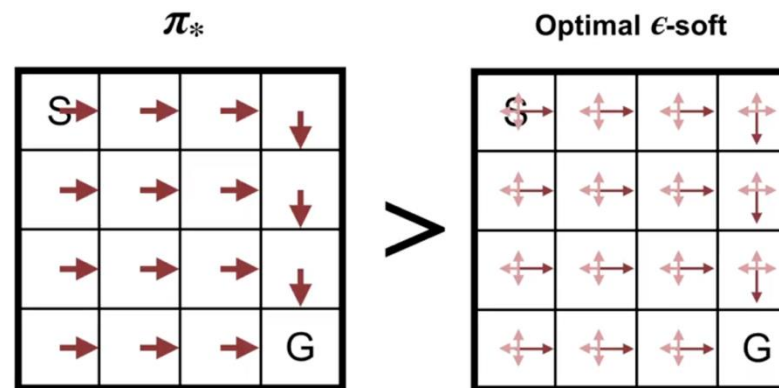


ϵ -Soft policies



~~Exploring Starts~~

ϵ -soft policies may not be optimal



On-Policy and Off-Policy

- **On-Policy**: improve and evaluate the policy being used to select actions
- **Off-Policy**: improve and evaluate a *different* policy from the one used to select actions

- **Off-policy learning** allows learning an optimal policy from suboptimal behavior
- Using importance sampling we can define Off-policy MC algorithm

Target Policy

$$\pi(a | s)$$

Behavior Policy

$$b(a | s)$$

- | | |
|--|--|
| <ul style="list-style-type: none"> • Learn values for this policy • For example the optimal policy | <ul style="list-style-type: none"> • Select actions from this policy • Generally an exploratory policy |
|--|--|

Target policy:
The policy is being learned about

Behavior policy:
A policy is used to generate behavior

Off-Policy Learning & Importance Sampling

Learning from the behavior of others.

Target Policy (π): The optimal path we want to evaluate.

Behavior Policy (b): The exploratory path we actually took to gather data.

$$\rho_{t:T-1} = \prod \left[\frac{\pi(A_k | S_k)}{b(A_k | S_k)} \right]$$

Target Policy Probabilities
Behavior Policy Probabilities

The Coverage Assumption

$\pi(a|s) > 0$ implies $b(a|s) > 0$ The behavior policy must have a non-zero chance of taking every action that the target policy might take.

This ratio mathematically transforms the returns observed under policy b so they reflect the correct expected value under policy π .

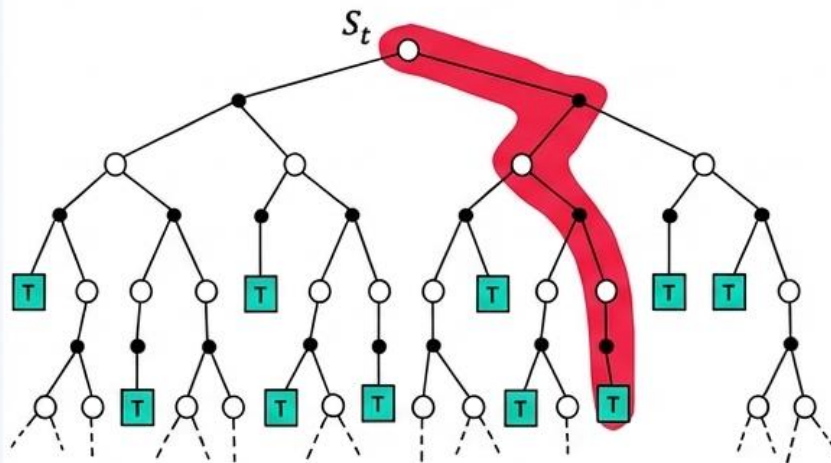
On-Policy	Off-Policy
Simpler	More complex
Faster convergence	Greater variance Slower convergence
	More general, more powerful When Target policy = Behavior policy it is an On-policy
	Can learn from data generated by a conventional non-learning controller or from a human

Temporal Difference

The Breakthrough: Temporal Difference (TD) Learning

Monte Carlo: Hindsight

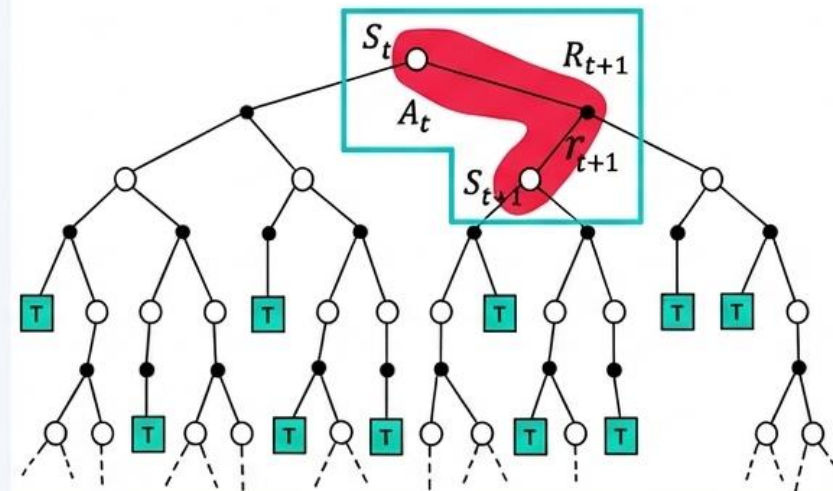
$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$



- Must wait for the episode outcome.
- No Bootstrapping.

Temporal Difference: Real-Time

$$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$



- Learns online, step-by-step.
- Bootstraps: Updates estimates based on other estimates.

TD methods do not require waiting for the end of an episode and naturally handle non-stationary, continuous environments.

- Simple every-visit Monte Carlo method suitable for nonstationary environments

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

MC methods must wait until the end of the episode to determine the increment

- Simple TD method update

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

TD methods need to wait only until the next time step

Deconstructing TD(0) Prediction

$$V(S_t) \leftarrow V(S_t) + \alpha [\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD Target}} - \underbrace{V(S_t)}_{\text{Current Estimate}}]$$

Current Estimate

Our existing guess of the state's value.

Learning Rate

How aggressively we update our belief.

TD Target

Our new, slightly better guess based on one immediate real reward plus our discounted estimate of the next state.

TD Error (δ_t)

The mathematical representation of surprise. The difference between what we expected to happen and what we actually observed.

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated

Algorithm parameter: step size $\alpha \in (0, 1]$

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

$A \leftarrow$ action given by π for S

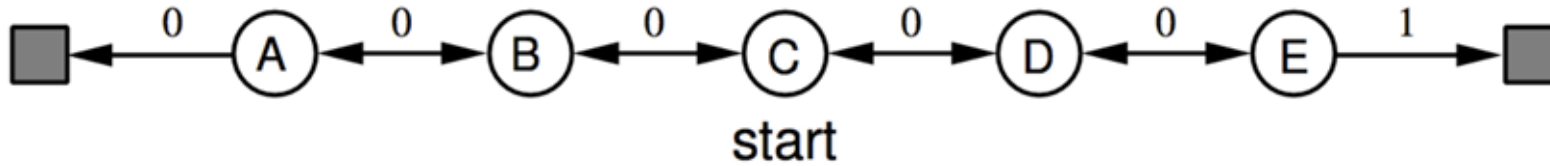
 Take action A , observe R, S'

$V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$

$S \leftarrow S'$

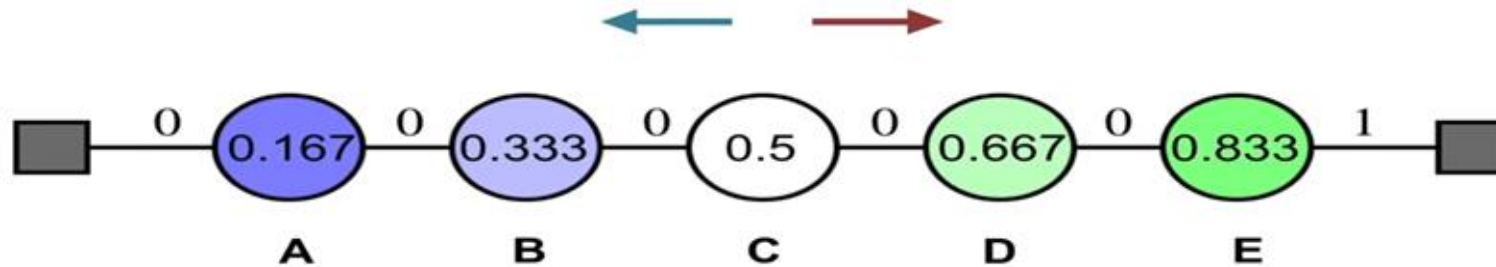
 until S is terminal

Example: Random Walk



$$\pi(. | s) = 1/2 \quad \forall s \in \mathcal{S} \quad \gamma = 1$$

- True/Target Values

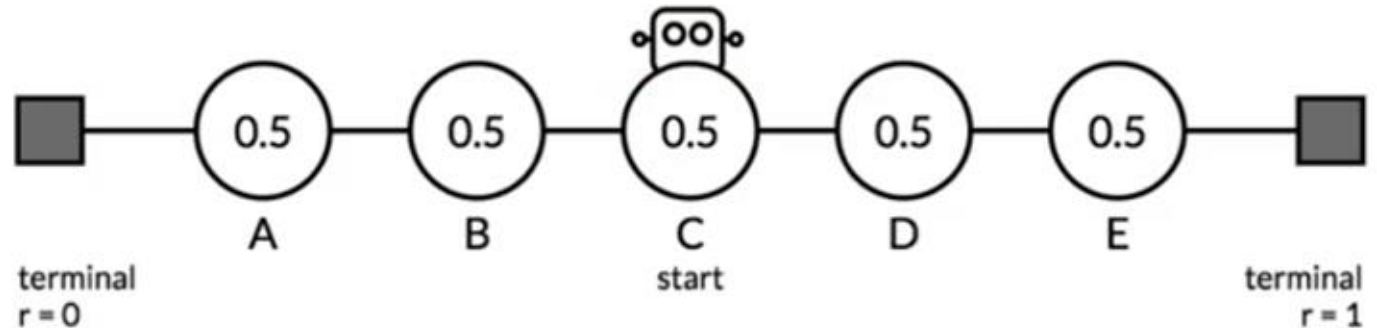


Example: Random Walk

- Initialize the state-values for MC and TD methods

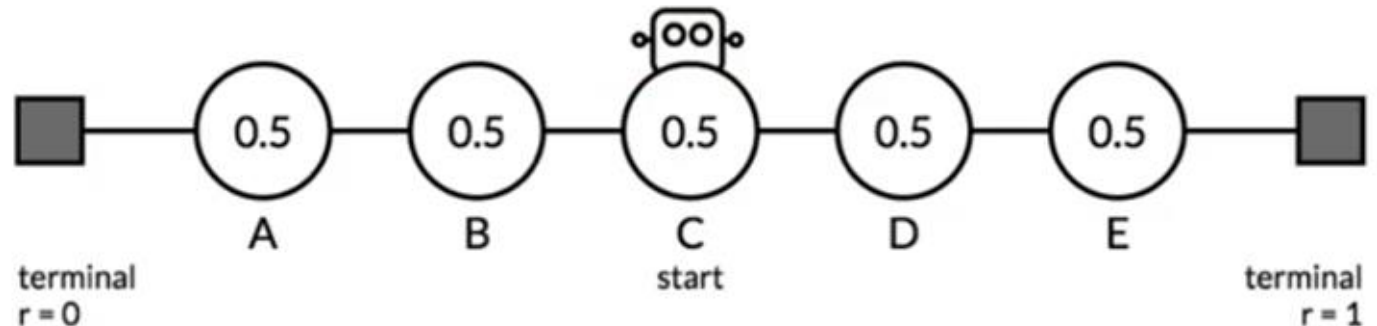
$$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

TD learning



$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$

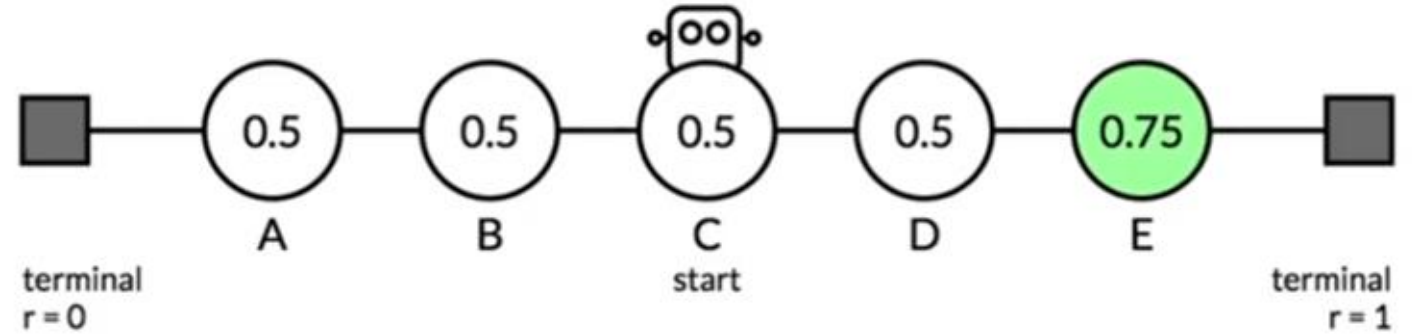
MC Learning



- After episode 1

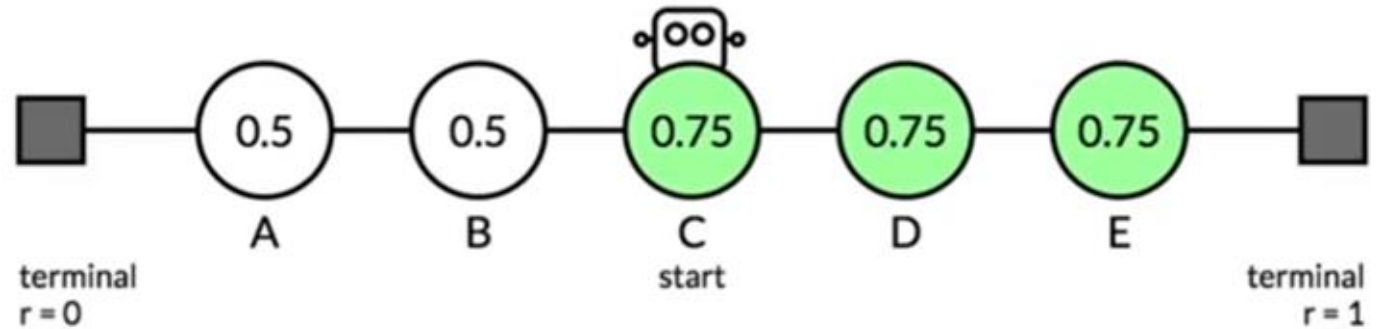
$$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

TD learning



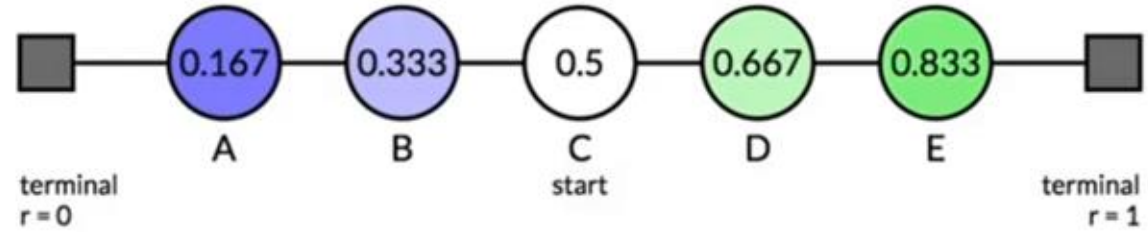
$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$

MC Learning

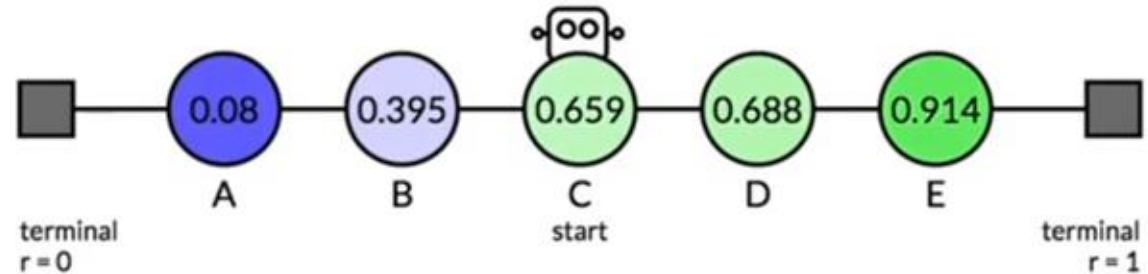


Example: Random Walk

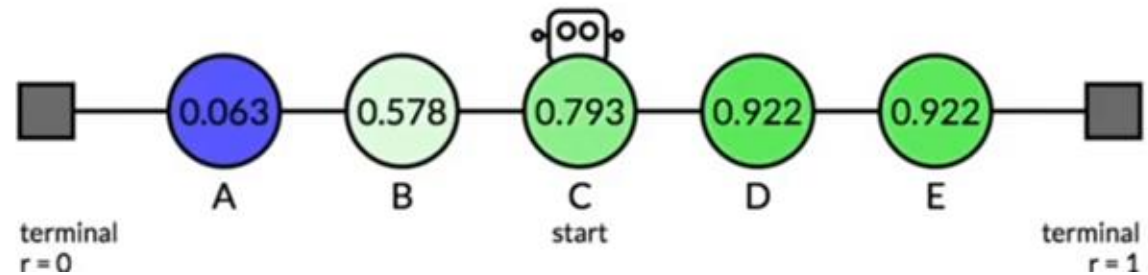
- After multiple episodes



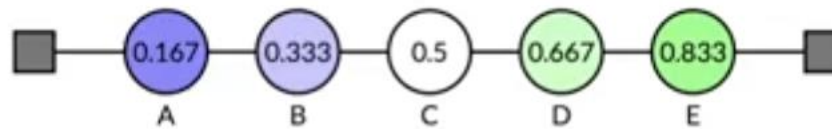
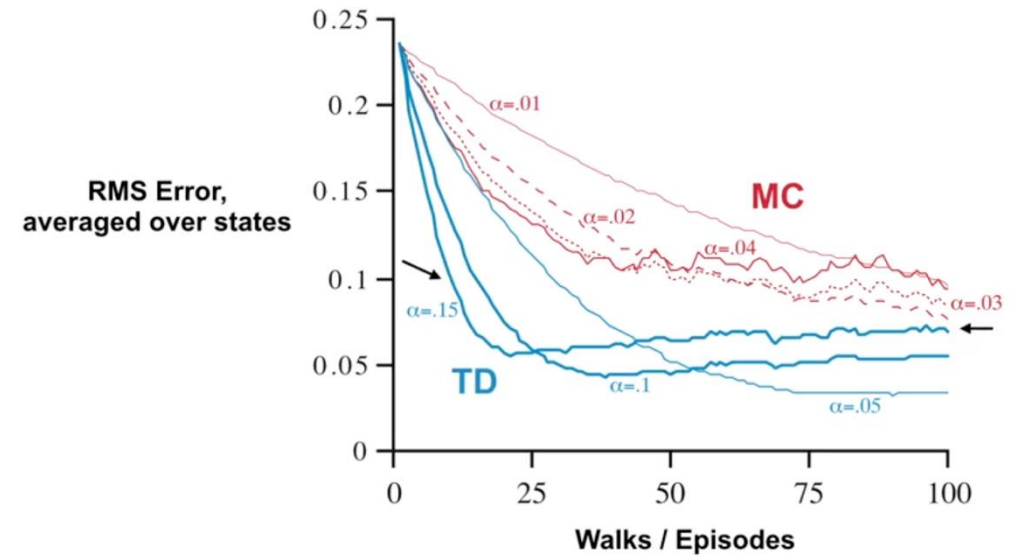
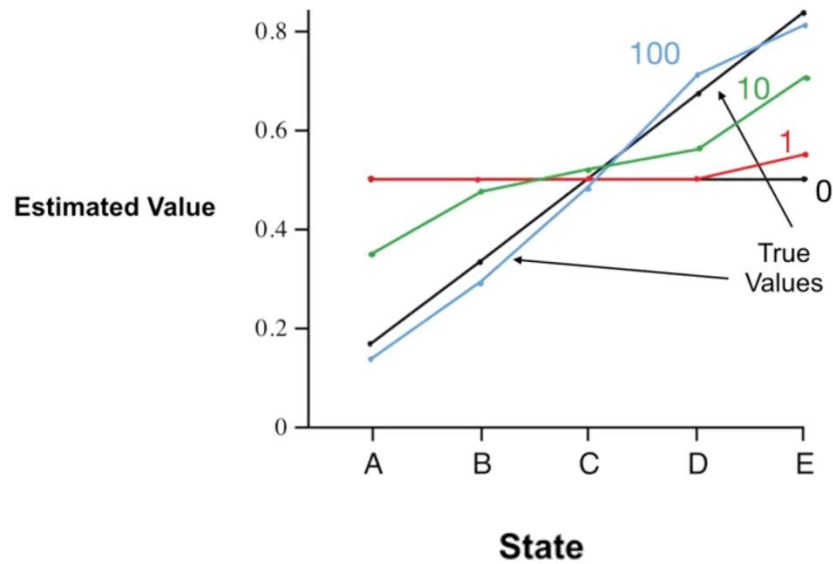
Updates using TD Learning



Updates using Monte Carlo



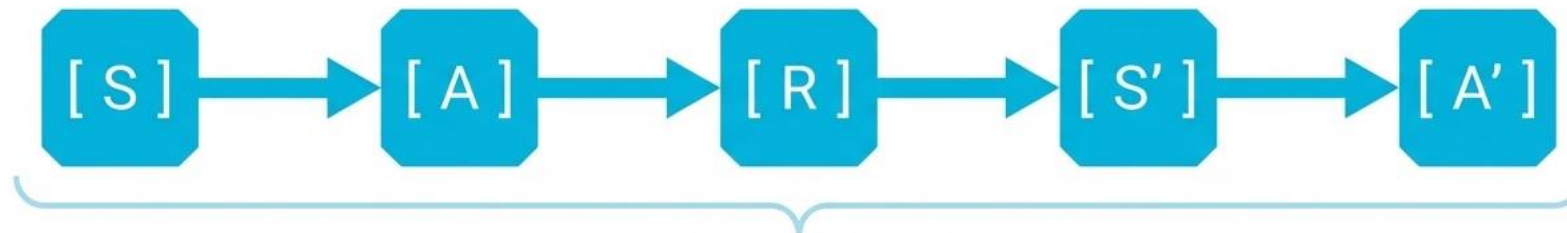
TD converges faster to a lower final error in this problem.



TD for Control: SARSA

SARSA: On-Policy Control

Learning while following the current policy.



The SARSA Quintuple

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

Recall: $V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$

On-Policy Nature

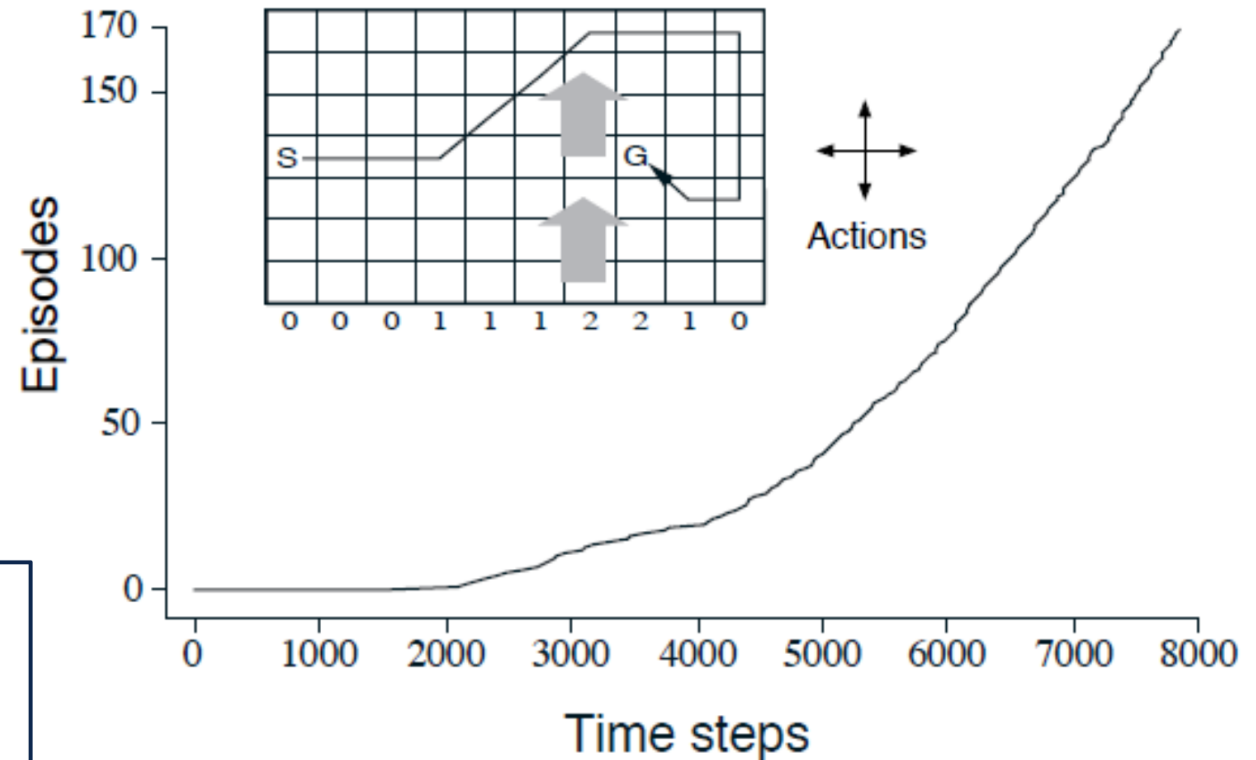
SARSA evaluates the exact policy it is currently following. The update requires knowing the actual next action (A') that the agent takes, tightly coupling exploration mistakes into the value estimate.

Windy gridworld

- undiscounted
- episodic task
- constant rewards of -1 until the goal state is reached
- 4 actions

Note:

MC methods cannot easily be used on this task because termination is not guaranteed for all policies. If a policy was ever found that caused the agent to stay in the same state, then the next episode would never end



← Sarsa do not have this problem

Q-learning: Off-policy TD Control

- Sarsa

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[\underbrace{R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})}_{\text{Sarsa}} - Q(S_t, A_t) \right]$$

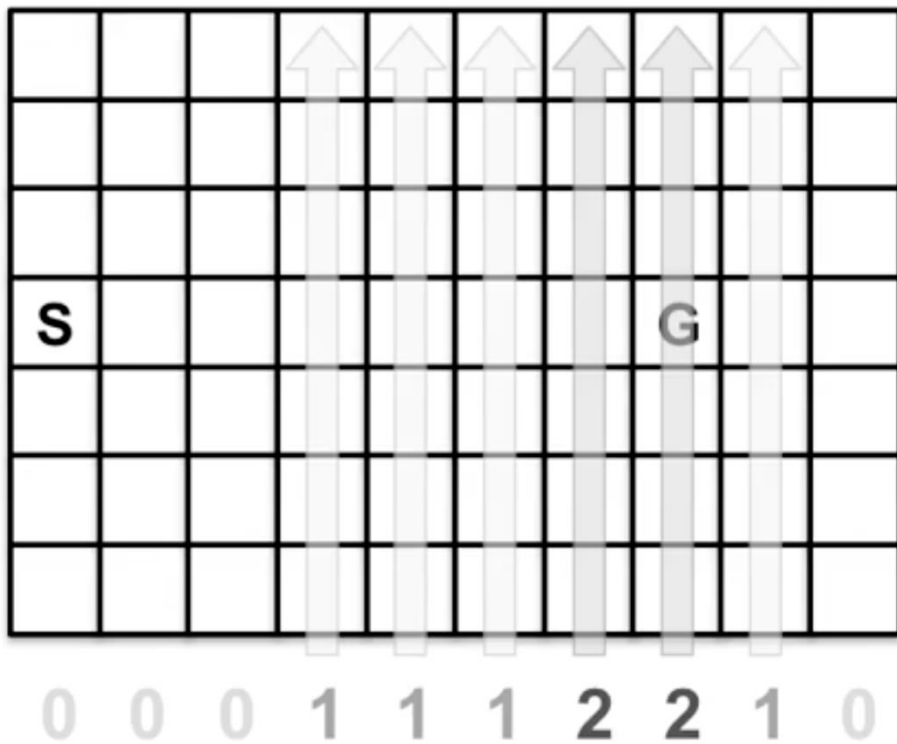
- Q-learning

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[\underbrace{R_{t+1} + \gamma \max_a Q(S_{t+1}, a)}_{\text{Q-learning}} - Q(S_t, A_t) \right]$$

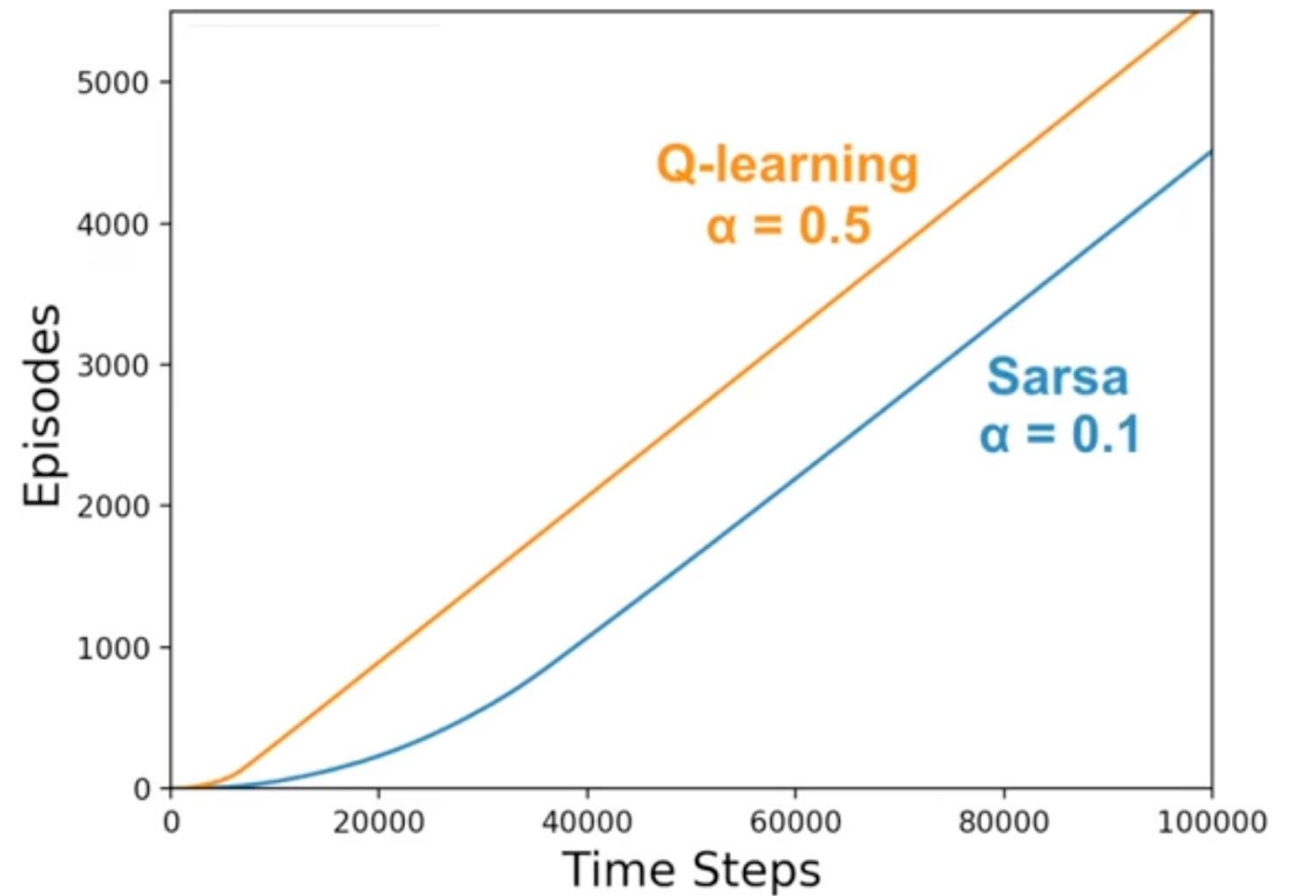
- The learned action-value function directly approximates q_*
- Independent of the policy being followed
- Minimal requirement of convergence:
 - all state-action pairs continue to be updated

- **The Concept:** Q-learning evaluates the optimal policy while behaving according to a separate exploratory (e.g., ϵ -greedy) policy.
- **The Positive Bias Warning:** Because it inherently assumes the maximum estimated value, it can suffer from **positive bias** (overestimating action values) in noisy environments.

The Windy Gridworld



$\epsilon = 0.1$



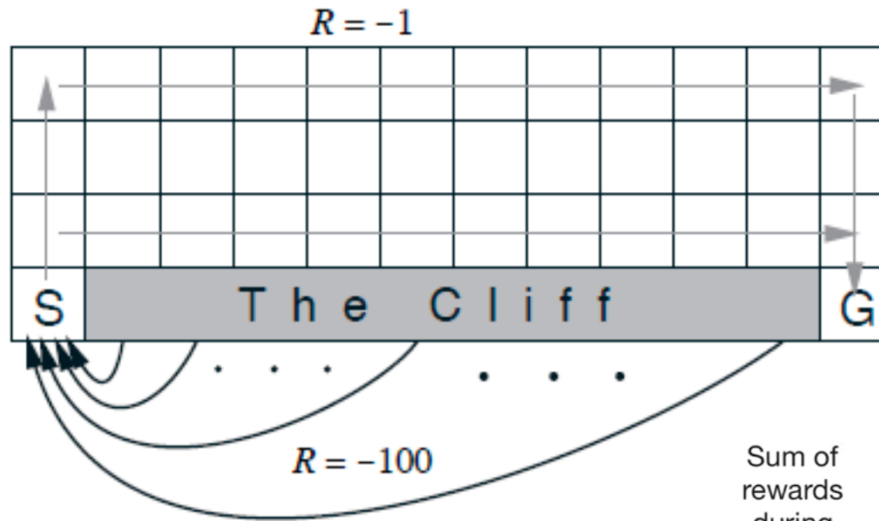
Cliff Walking example (Sarsa vs Q-learning)

Sarsa

safe path

optimal path

Q-learning



- Q-learning is **off-policy** without using **importance sampling**
- Learning **on-policy** or **off-policy** may perform differently in **control**

Q-learning: learns values for the optimal policy. This results in its occasionally falling off the cliff because of the ϵ -greedy action selection.

Sarsa: takes the action selection into account and learns the longer but safer path

Sum of rewards during episode



---- Q-Learning Path

Optimal but Risky. Learns the shortest theoretical route, completely ignoring the random exploration noise that occasionally makes it fall off.

— SARSA Path

Sub-optimal but Safe. Accounts for its own random exploration mistakes, taking a wider arc to avoid catastrophic falls.

Expected SARSA

Expected SARSA

Reducing the variance of random next-action sampling.



$$\text{TD Target: } R_{t+1} + \gamma \sum_a \pi(a|S_{t+1})Q(S_{t+1}, a)$$

Instead of taking the absolute maximum (Q-learning) or flipping a coin to sample a specific next action (SARSA), **Expected SARSA** mathematically calculates the **expected value of the next state** based on the current policy probabilities.

The Benefit: Eliminates the variance inherent in Sarsa's random actions while safely allowing **off-policy** learning.

NotebookLM

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma \sum_{a'} \pi(a' | S_{t+1}) Q(S_{t+1}, a') - Q(S_t, A_t) \right)$$

- The **Expected Sarsa** algorithm explicitly computes the **expectation under its policy**, which is more expensive than sampling but has **lower variance**

Synthesis

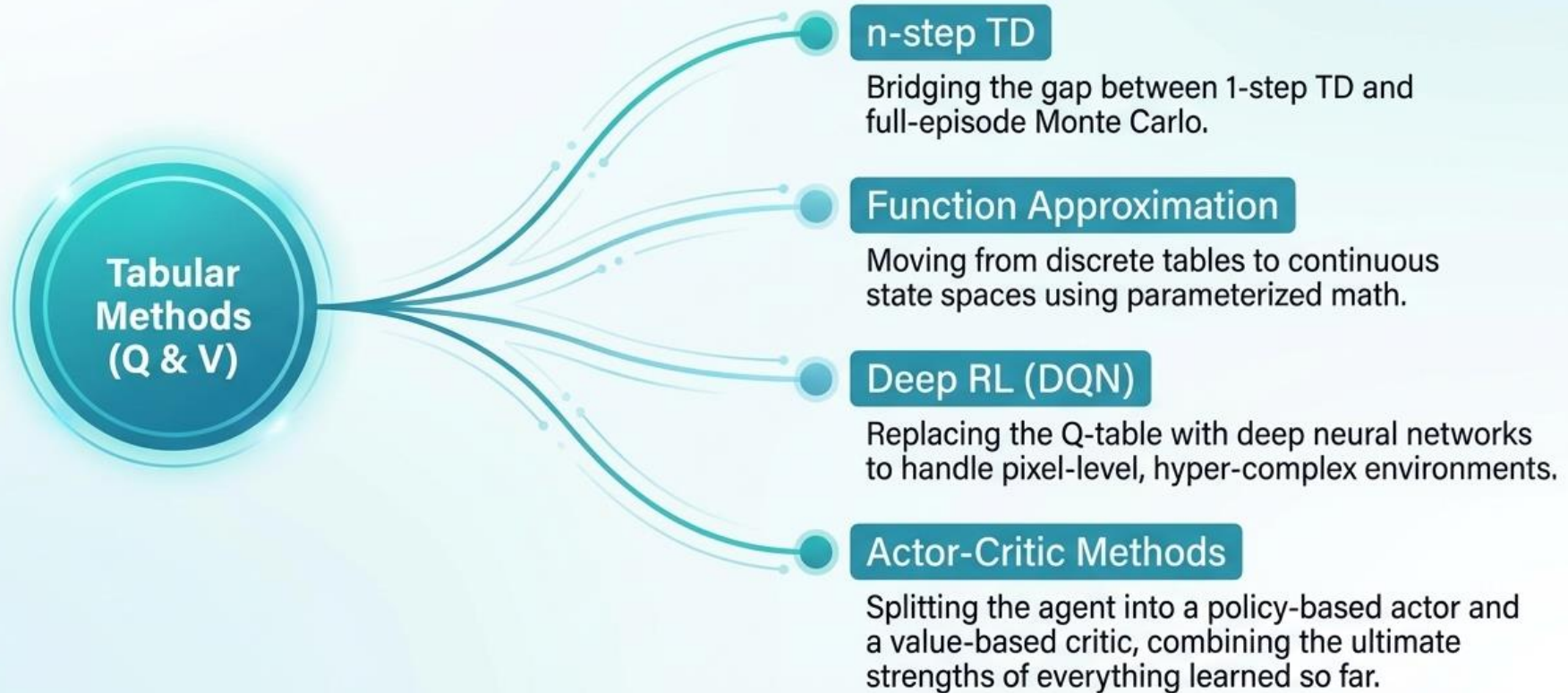
Synthesis: The Landscape of Tabular Methods

Algorithm	Model Needed?	Bootstrapping?	Update Type	Online Learning?
DP	Yes	Yes	Expected	No
MC	No (Raw Experience)	No (Waits for outcome)	Sample	No (End of episode)
TD	No (Raw Experience)	Yes (From estimates)	Sample	Yes (Step-by-step)

Conclusion: **TD** combines the **sampling** of MC with the **bootstrapping** of DP, resulting in the most flexible model-free framework for real-world environments.

Future Directions

The blueprint of experience scales infinitely.



- [1] Sutton, R. S. and Barto, A. G. (2018), Reinforcement Learning: An Introduction (Second edition). MIT Press
- [2] Adam White and Martha White, Reinforcement Learning Specialization, University of Alberta, Coursera